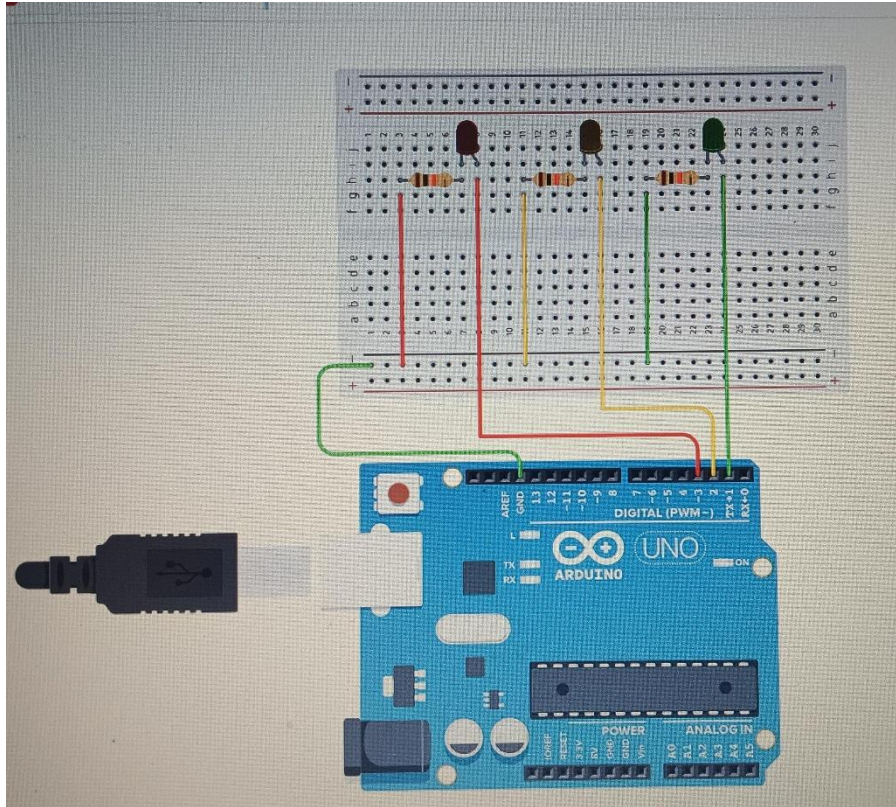


TRAFFIC LIGHT CONTROL SYSTEM USING TINKER CAD



DESCRIPTION

This project simulates a real-world traffic light system using an Arduino Uno, three LEDs (red, yellow, green), and resistors on a breadboard. The Arduino controls the LEDs to cycle through red (stop), yellow (caution), and green (go) phases with programmed delays, mimicking traffic signal timing. Users connect the components as shown in the attached image: LEDs with anodes to digital pins (likely pins 9, 10, 11 for D1-D3), cathodes through 1K resistors to ground, and Arduino for power and control. This beginner-friendly setup introduces digital output, timing, and sequencing without complex sensors.

Components Explanation

- **D1: Red LED** (Quantity: 1):

Emits red light to signal "stop." Connected to an Arduino digital pin; lights up first for 3-5 seconds. LEDs have anode (longer leg) to pin and cathode to resistor.

- **D2: Yellow LED** (Quantity: 1):

Provides caution before green or after red. Activates briefly (1-2 seconds) to warn of changes; same wiring as red LED.

- **D3: Green LED** (Quantity: 1):

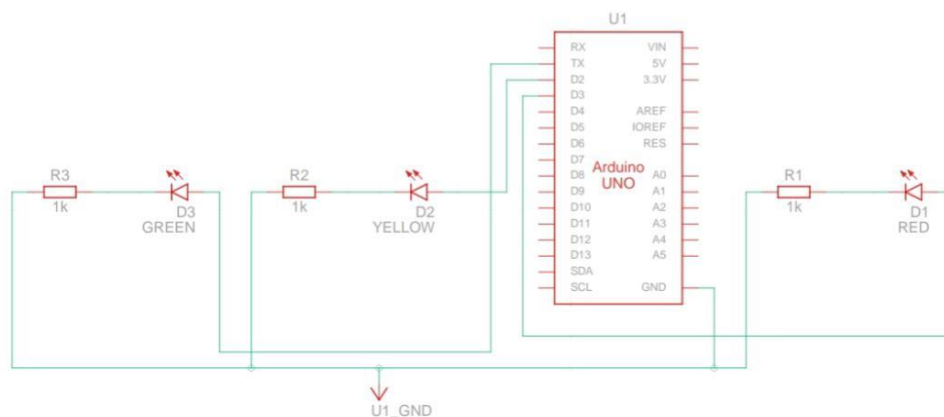
Indicates "go." Stays on longest (5-10 seconds); wired identically to others for balanced current limiting.

- **R1: 1K Resistor** (Quantity: 3):

Limits current to protect LEDs (typically 20mA max). One per LED, placed between cathode and ground to drop voltage (e.g., from 5V Arduino supply).

- **U1: Arduino Uno R3** (Quantity: 1):

Microcontroller brain. Powers the circuit (5V/GND), runs code to set pin HIGH/LOW for LED states, and uses delay() for timing loops.



Circuit Operation

In this circuit, an Arduino Uno controls three LEDs—red (D1), yellow (D2), and green (D3)—each in series with a 1 k Ω resistor to limit current before going to ground. The LED anodes connect to three digital pins; when a pin goes HIGH, current flows through that LED and it turns on. The program turns on the red LED first to indicate stop, then switches to yellow as a warning, then to green to indicate go, and finally back to yellow before returning to red. Delays in the code set how long each light stays on, so the full traffic pattern repeats automatically.

OUTCOMES/LEARNINGS

From this traffic-light circuit, I learned how to wire and protect LEDs using series 1 k Ω resistors so they can be safely driven from the Arduino. I understood LED polarity (anode to digital pin, cathode to resistor and ground) and how wrong orientation stops current flow.

I practiced using digital pins as outputs and controlling them with simple commands like `pinMode` and `digitalWrite` to turn each color on and off. I saw how `delay` creates timing, so the red, yellow, and green LEDs follow a realistic sequence. I also learned to read a basic circuit diagram and match it to breadboard connections.

While testing, I developed debugging skills by fixing issues such as LEDs not lighting due to incorrect wiring. Overall, the project showed how code and hardware work together to create a small, automatic traffic-light system.